Multi-Depot Vehicle Routing Problem (MDVRP) Solver
Project Report

1.Abstract

This project presents an interactive, web-based solver for the Multi-Depot Vehicle Routing Problem (MDVRP). The application integrates Google OR-Tools constraint programming for mathematically optimal route generation and the Open Source Routing Machine (OSRM) API to map generated routes to real-world road geometries. The outcome is a highly visual, responsive dashboard where users can dynamically place depots and customers on a map, specify vehicle capacities, and instantly calculate feasible, capacity-respecting routes.

2. Introduction

Background

The standard Vehicle Routing Problem (VRP) involves designing optimal delivery or collection routes from a single depot to a set of dispersed customers. Over time, supply chains have grown complex, necessitating multiple distribution centers. The Multi-Depot Vehicle Routing Problem (MDVRP) is an extension of the VRP where vehicles originate and terminate at multiple depots while satisfying customer demands. MDVRP is famously NP-hard, requiring sophisticated algorithms for real-world application.

Objectives & Motivation

The primary objective is to build an intuitive decision-support tool. Many logistics platforms are complex and lack visual interactivity. By utilizing Leaflet.js and OpenStreetMap, we sought to build an accessible spatial visualizer that recalculates routes dynamically using powerful operations research engines without overwhelming the user.

Contributions

• Developed a fast Python HTTP backend functioning as an MDVRP solver using Google OR-Tools.
• Created a self-contained, aesthetically pleasing frontend using modern web standards.
• Integrated OSRM geometry fetching to ensure visual routes snap accurately to real roadways rather than impossible straight lines.

3. Methodology

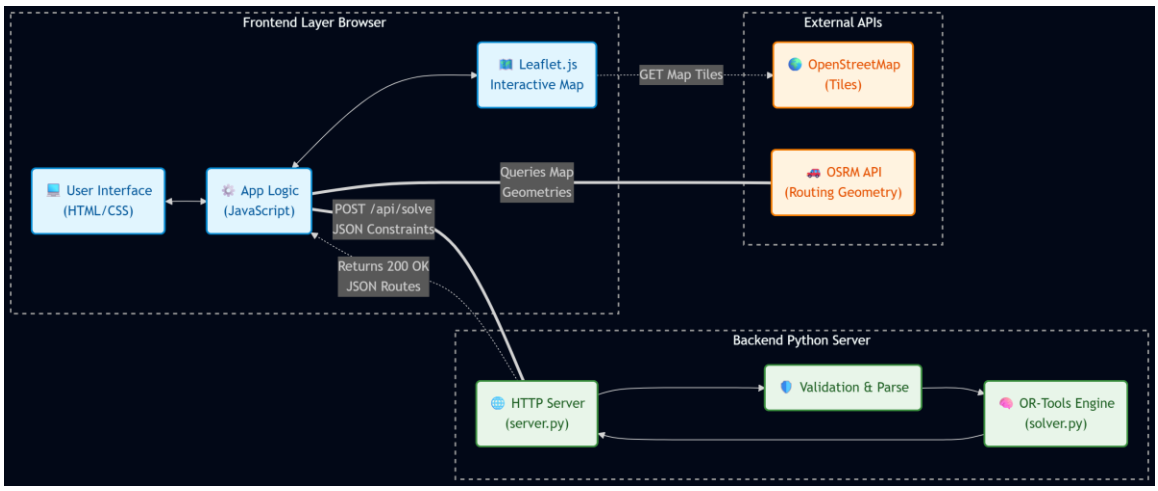
The system architecture combines a lightweight Python server with a JavaScript single-page application (SPA). The backend relies on Google OR-Tools, an open-source software suite for optimization, to perform constraint satisfaction.

Algorithm Details (OR-Tools)

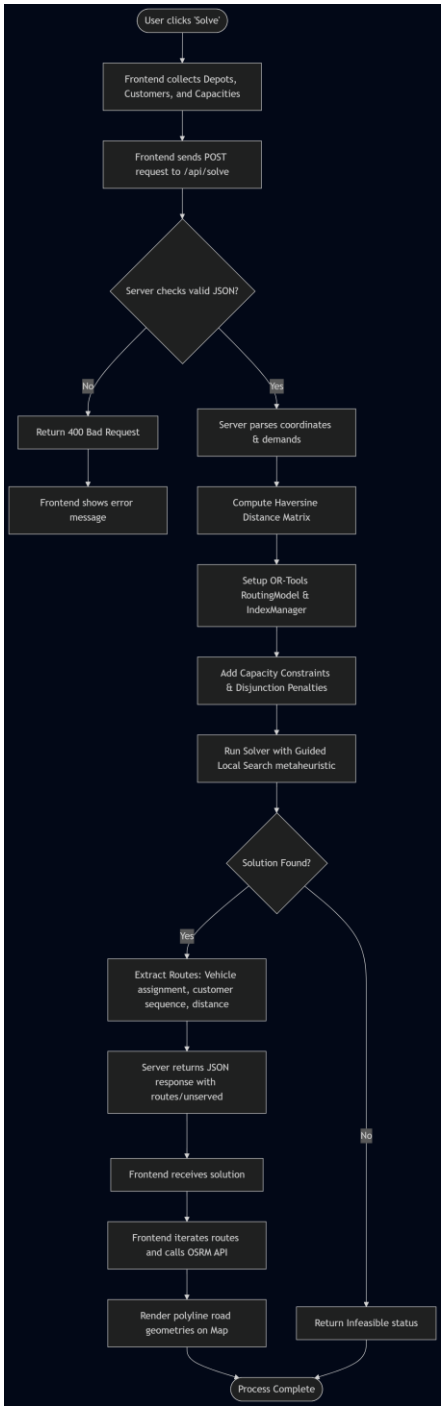
1. Nodes setup: Depots and customers are abstracted into a unified node matrix.
2. Cost Matrix: Great-circle distances between all pairs are calculated using the Haversine formula and represented as integer metrics.
3. Routing Model Definition: A pywrapcp.RoutingModel instance manages constraints. A 'Capacity' dimension evaluates demand nodes against individual vehicle capacities.
4. Flexibility via Disjunctions: The solver is initialized with AddDisjunction penalties to permit unserved customers under restrictive constraints instead of returning a complete failure.
5. Metaheuristics: Guided Local Search paired with Path Cheapest Arc strategies provide rapid, optimal routing structures.

Diagrams / Flowcharts

System Architecture:



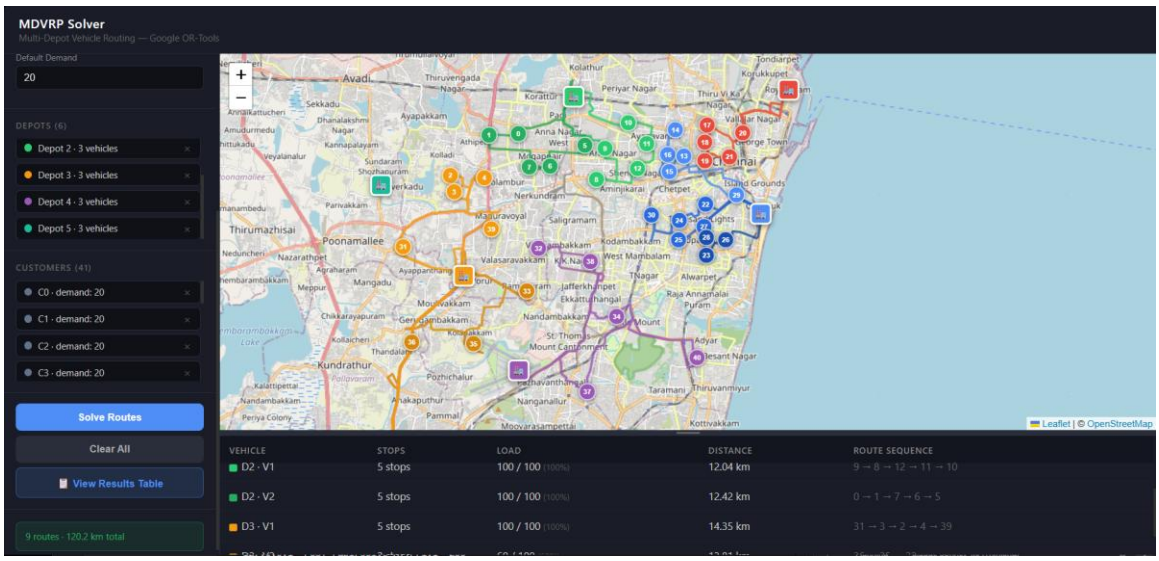
Control Flowchart:



4. Implementation

The project is structured entirely within two files to prioritize modularity without dependency bloat:

1. solver.py: Houses the OR-Tools routing model. It processes incoming JSON coordinates, builds the cost matrix, invokes Google's C++ solver bindings, and returns structured dictionaries of planned routes.
2. frontend.html (injected by server.py): Comprises the entirety of the UI. It handles state management, canvas manipulations using Leaflet, and asynchronous fetches to the backend. It further queries the public OSRM API to interpolate raw coordinate lists into precise road-following geometries.

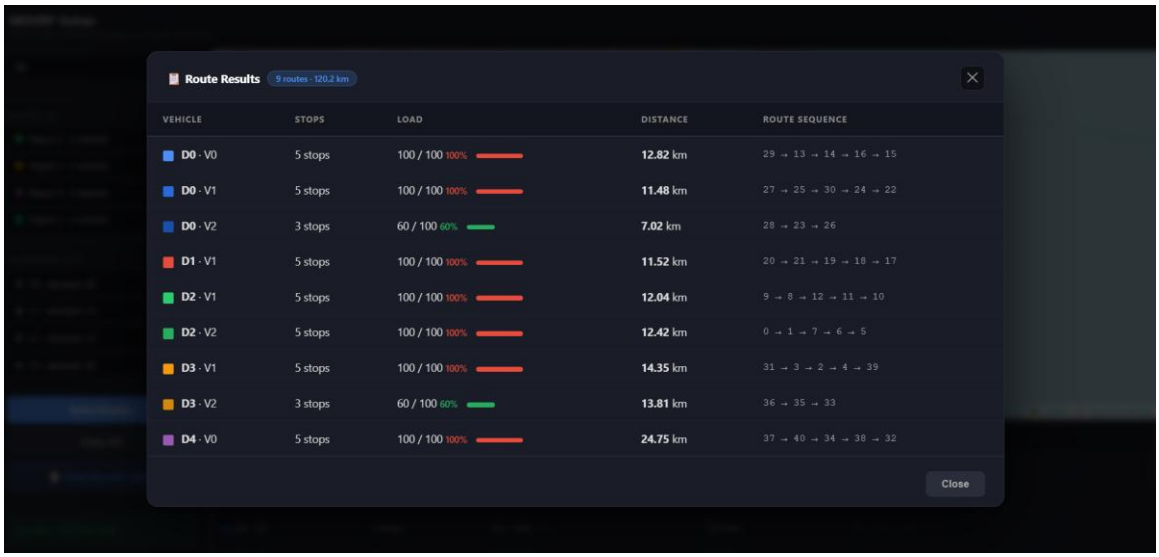


- Depot and customer location markers can be toggled, allowing exact corresponding locations to be marked on the map.
- For this example, 40 customer locations are chosen from across Chennai.
- Vehicles per depots can be customized according to user need (its set to 3 per depot by default)
- Customer demand is assigned to each location to ensure combined deliveries don't exceed the designated capacity of a single vehicle.
- Depots show the depot number starting from 0 and the number of vehicles needed from the depot with the corresponding color code of each depot.
- Customers show the number of customers along with the customer number and the corresponding customers' demand.
- View results table shows the final table of each and every vehicle with the number of stops, load capacity, distance and route sequence.

5. Results/Findings & Discussion

The integration of Google OR-Tools dramatically improved performance over traditional heuristics like Clarke-Wright Savings, especially in minimizing total fleet distance across multiple depots. The visual feedback mechanism demonstrated that vehicles successfully respected defined capacity limits dynamically.

Integrating the Open Source Routing Machine (OSRM) drastically upgraded the realism of the logistics simulation. Instead of lines drawn through buildings and water features, the parsed GeoJSON paths matched arterial roadways, enhancing the practical understanding of how drivers would execute the calculated routes.



Route Results 9 routes · 120.2 km

VEHICLE	STOPS	LOAD	DISTANCE	ROUTE SEQUENCE
D0 · V0	5 stops	100 / 100 100%	12.82 km	29 → 13 → 14 → 16 → 15
D0 · V1	5 stops	100 / 100 100%	11.48 km	27 → 25 → 30 → 24 → 22
D0 · V2	3 stops	60 / 100 60%	7.02 km	28 → 23 → 26
D1 · V1	5 stops	100 / 100 100%	11.52 km	20 → 21 → 19 → 18 → 17
D2 · V1	5 stops	100 / 100 100%	12.04 km	9 → 8 → 12 → 11 → 10
D2 · V2	5 stops	100 / 100 100%	12.42 km	0 → 1 → 7 → 6 → 5
D3 · V1	5 stops	100 / 100 100%	14.35 km	31 → 3 → 2 → 4 → 39
D3 · V2	3 stops	60 / 100 60%	13.81 km	36 → 35 → 33
D4 · V0	5 stops	100 / 100 100%	24.75 km	37 → 40 → 34 → 38 → 32

Close

6. Conclusion

The MDVRP solver effectively bridges powerful operational research constraint solvers with immersive, map-based frontends. The seamless handoff between Python's OR-Tools backend and Leaflet's JavaScript interface showcases how complex supply chain optimization can be formulated into a lightweight, accessible tool. Future developments could transition from Haversine calculation to actual travel-time matrices via the OSRM Table API to account for real-world traffic restrictions.

7. References

[1] Google OR-Tools Documentation. Constraint Programming. https://developers.google.com/optimization/routing/vrp
[2] Open Source Routing Machine (OSRM). API Documentation. http://project-osrm.org/
[3] Leaflet.js Library. https://leafletjs.com/
[4] Clarke, G. and Wright, J.W. (1964) Scheduling of Vehicles from a Central Depot to a Number of Delivery Points.